21 min read

Bellman Equations and Dynamic Programming

RL Part 3: Bellman expectation and optimality equations, policy iteration, value iteration, and why dynamic programming needs a model.

Recap

In the previous chapter, we formalized the agent-environment interaction as a Markov decision process (MDP).

We began with the Markov property, which states that the future depends on the past only through the present state. This is the assumption that makes the entire framework tractable: once you know the current state, you can discard the history.

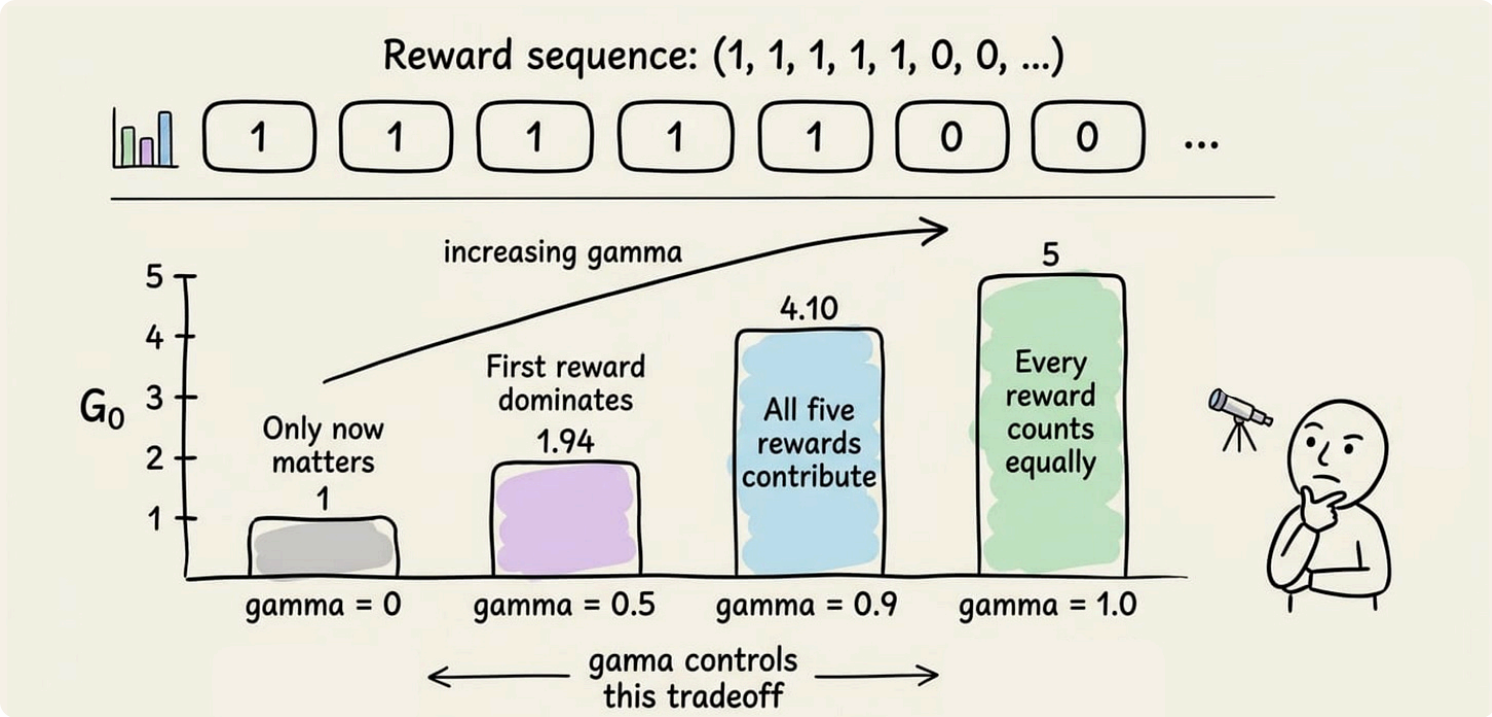
$$\Pr(S_{t+1} = s' \mid S_t, A_t, S_{t-1}, A_{t-1}, \ldots, S_0, A_0) = \Pr(S_{t+1} = s' \mid S_t, A_t)$$

Distribution of the next state given the entire history

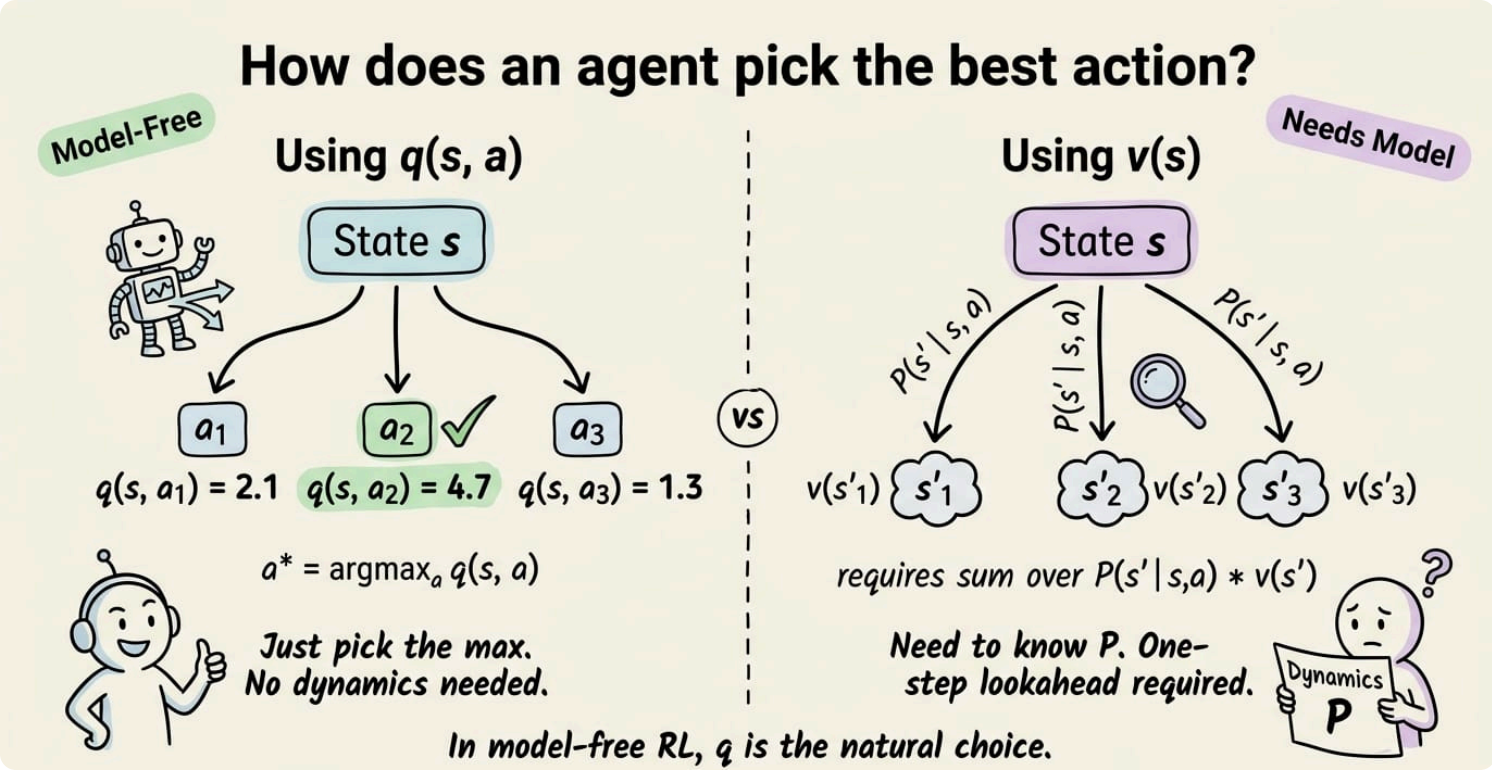
Distribution of the next state based on the current state/action

We then defined the MDP as a 5-tuple (S, A, P, R, γ): states, actions, a transition function, a reward function, and a discount factor.

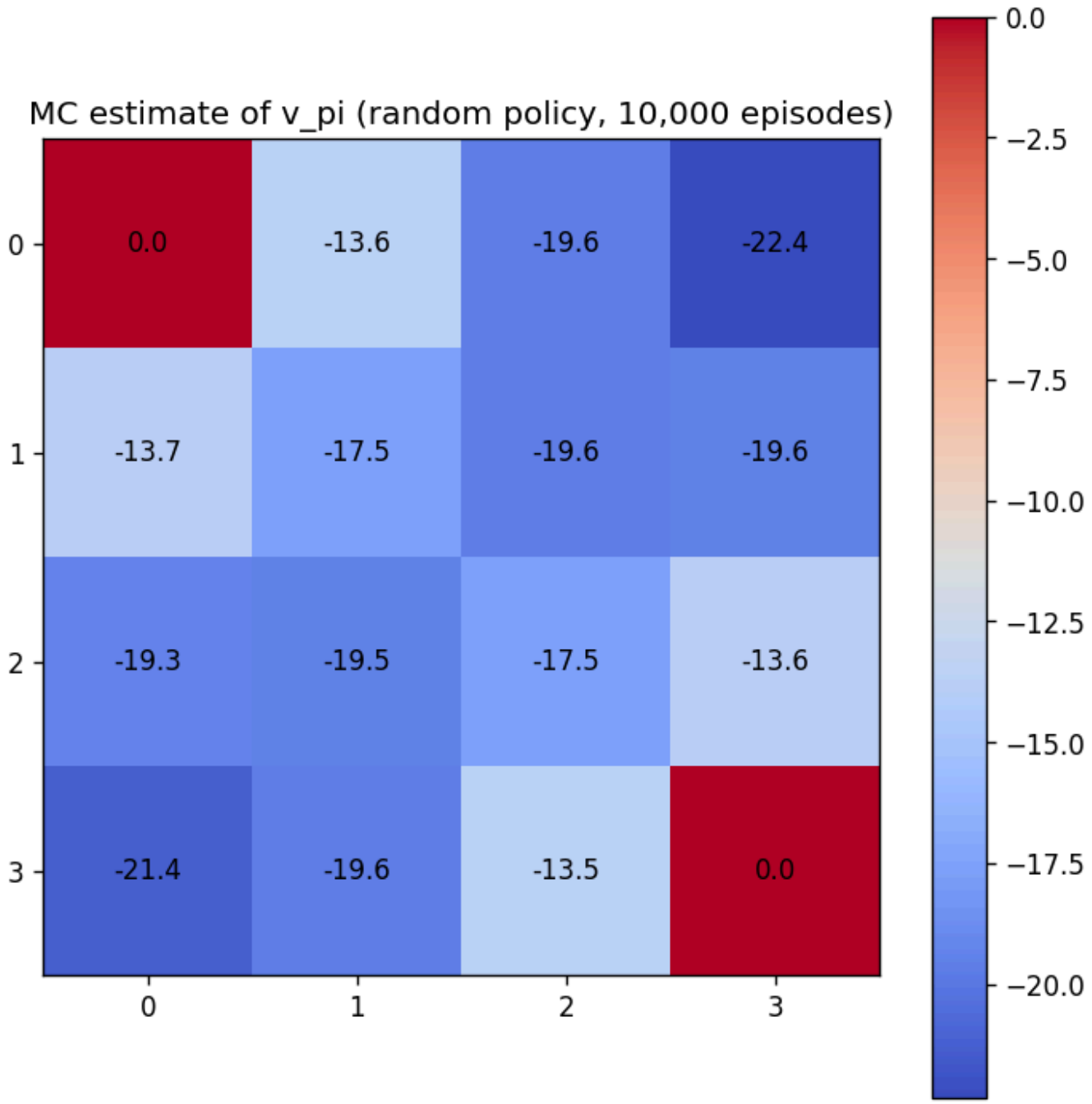
We discussed episodic versus continuing tasks. We defined the return G_t as the discounted cumulative reward, and explained how γ controls the agent's far-sightedness.



We introduced policies, both deterministic and stochastic, as the object the agent learns. We defined the state-value function $v_\pi(s)$ as the expected return from state s under policy π, and the action-value function $q_\pi(s, a)$ as the expected return from taking action a in state s and then following π.



Finally, we built a 4×4 gridworld and used Monte Carlo rollouts to estimate v_π for a random policy.

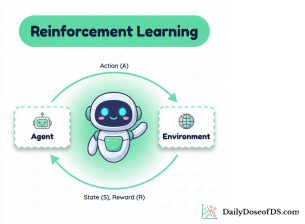

MC estimate of v_pi (random policy, 10,000 episodes)
0.0
-13.6
-19.6
-22.4
-13.7
-17.5
-19.6
-19.6
-19.3
-19.5
-17.5
-13.6
-21.4
-19.6
-13.5
0.0
0
1
2
3
0.0
−2.5
−5.0
−7.5
−10.0
−12.5
−15.0
−17.5
−20.0

If you have not read Chapter 2, we recommend doing so first:

Markov Decision Processes and Value Functions

RL Part 2: Markov decision processes, returns, policies, and value functions.

Daily Dose of Data Science • Avi Chawla



Introduction

In Chapter 2, we computed v_π by running thousands of episodes and averaging the returns.

Recall that $v_\pi(s)$ tells us how much total reward the agent can expect to collect starting from state s if it follows policy π.

In Part 2, we estimated this the brute-force way. Drop the agent into a state, let it act until the episode ends, write down the total reward, and do that thousands of times. Average those numbers and you get $v_\pi(s)$.

Hands-on: gridworld and Monte Carlo policy evaluation

We have defined v_π as an expectation, but how do we actually compute it?

One natural answer: simulate many episodes under π, compute the return from each visited state, and average. The averaged return converges to v_π by the law of large numbers. This is called Monte Carlo (MC) policy evaluation.

The approach works, but it is expensive and noisy. Each estimate is a sample average, and the variance shrinks slowly.

There is a more direct route. The value functions satisfy recursive equations that relate the value of a state to the values of its successor states. These are the Bellman equations, named after Richard Bellman, who introduced the principle of optimality and the method of dynamic programming and compiled them into his landmark 1957 book "Dynamic Programming" (Princeton University Press).

Richard E. Bellman

The Bellman equations give us two things. First, they characterize v_π and q_π exactly, without simulation. Second, they characterize the optimal value functions v_* and q_*, which tell us the best possible performance in the environment.

Dynamic programming (DP) turns these equations into algorithms. Given a complete model of the environment (the transition function P and the reward function R), DP computes optimal policies exactly and efficiently for small problems.

In this chapter, we will derive the Bellman expectation equations for v_π and q_π, then the Bellman optimality equations for v_* and q_*.

We will also study four DP components: policy evaluation, policy improvement, policy iteration, and value iteration. We will close with a hands-on project, running both policy iteration and value iteration on the 4×4 gridworld and comparing the results side by side.

Let's begin!

The Bellman expectation equation for v_π

In Chapter 2, we established the recursive structure of the return:

$$G_t = R_{t+1} + \gamma G_{t+1}$$

If this looks unfamiliar, G_t is just the return, the total (discounted) reward the agent collects from time step t onward.

We defined it in Part 2 as described above. The recursive version above says that the total reward is whatever you get right now, plus γ times everything you get after that.

The state-value function is the expected return from state s under policy π:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

As we covered in Part 2, this expectation accounts for two kinds of randomness:

→ The agent might act randomly (because π can be stochastic)

→ The environment might respond randomly (because P can be stochastic).

The value $v_\pi(s)$ averages over both.

Substituting the recursive return into this expectation and expanding gives us the Bellman expectation equation for v_π. The derivation proceeds in a few steps:

1. Starting from the definition, we substitute the recursive return:

$$G_t = R_{t+1} + \gamma G_{t+1}$$

substitute

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma\, G_{t+1} \mid S_t = s]$$

2. Expectation is linear, so we can split it:

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s]$$

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} \mid S_t = s] + \gamma \mathbb{E}_\pi[G_{t+1} \mid S_t = s]$$

3. Now we need to expand these expectations. The agent is in state s and selects action a according to $\pi(a|s)$. The environment then transitions to state s' with probability $P(s'|s,a)$ and emits reward $R(s,a,s')$. Expanding over actions and next states:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) \Big[R(s,a,s') + \gamma\, v_\pi(s') \Big]$$

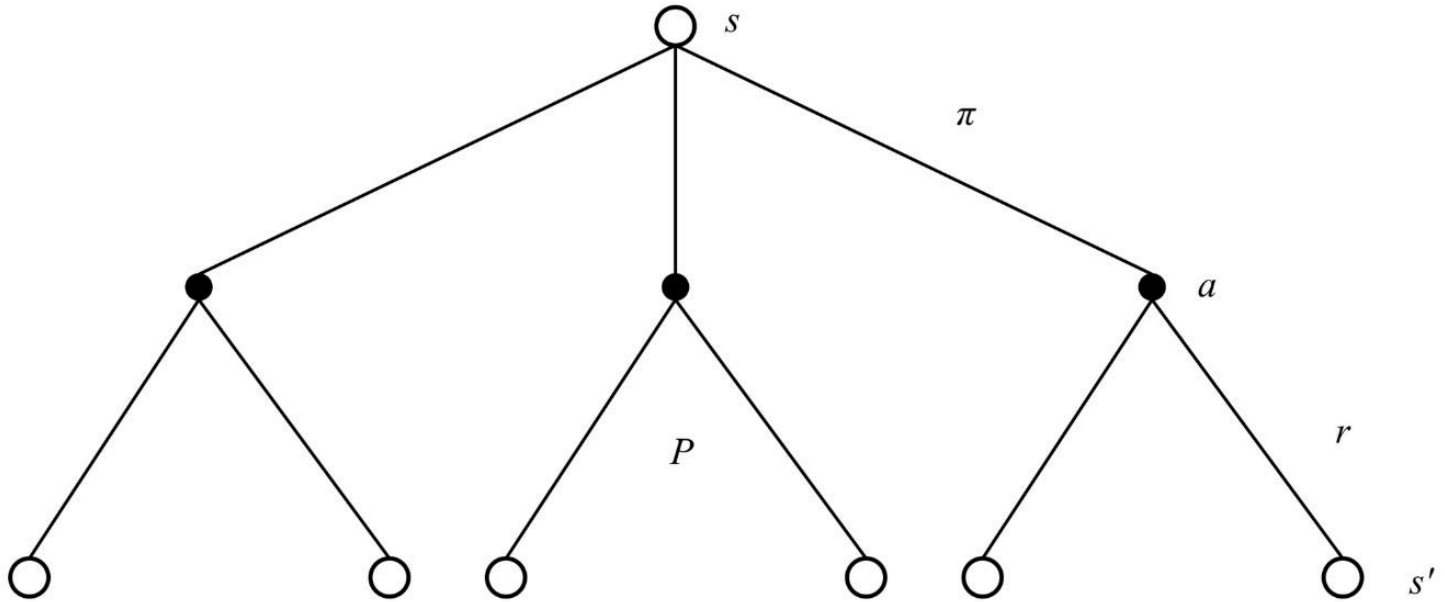
This is the Bellman expectation equation for v_π.

Let us unpack everything here:

- $v_\pi(s)$ is the value of state s under policy π.
- The outer sum runs over all actions a available in s, each weighted by $\pi(a|s)$, the probability the policy assigns to that action.
- The inner sum runs over all possible next states s', each weighted by $P(s'|s,a)$, the transition probability. Inside the brackets, $R(s,a,s')$ is the immediate reward for the transition, and $\gamma\, v_\pi(s')$ is the discounted value of the successor state.

> 👉 This equation requires knowing P and R.

The structure has two layers. The outer layer averages over the agent's action choice (governed by π). The inner layer averages over the environment's response (governed by P). The term in brackets combines the immediate reward with the future value, discounted by γ.



Backup diagram for v_π

The intuition is that the value of a state equals the average immediate reward plus the average discounted value of wherever you end up next, where "average" accounts for both the policy's randomness and the environment's randomness. If $\gamma = 0$, only the immediate reward matters. If $\gamma = 1$, the future matters as much as the present.

👉 Diagrams like the one just above are called backup diagrams. They show value information flow during learning updates. In other words, they

illustrate how information from future states (or state-action pairs) is passed back to earlier states when the agent updates its estimates.

Open circles = states, filled circles = state-action pairs, lines = actions from a state, or next-states from an action.

A concrete example

Consider a tiny two-state MDP to see the equation at work:

State A has two actions:

- go-left (stays in A)
- go-right (moves to terminal state B).

The policy π is: $\pi(\text{left}|A) = 0.5$, $\pi(\text{right}|A) = 0.5$. Every transition gives a reward -1. Transitions are deterministic. The discount factor is $\gamma = 0.9$.

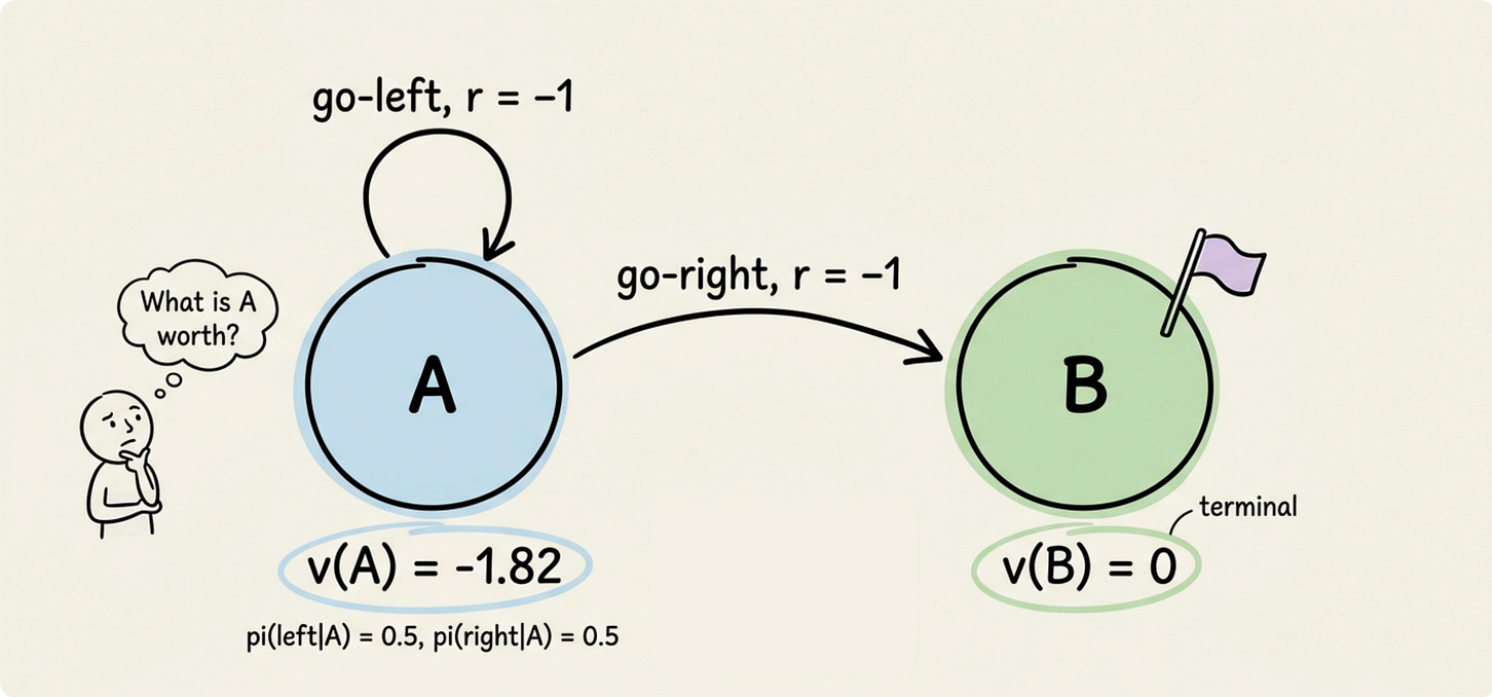
Since B is terminal, $v_\pi(B) = 0$. For state A, the Bellman equation gives:

$$v_\pi(A) = 0.5 \times [-1 + 0.9 \times v_\pi(A)] + 0.5 \times [-1 + 0.9 \times 0]$$

The first term covers go-left: reward -1, then back to A. The second covers go-right: reward -1, then terminal. Solving:

$$\begin{aligned} v_\pi(A) &= -0.5 - 0.5 + 0.45\, v_\pi(A) \\ 0.55\, v_\pi(A) &= -1 \\ v_\pi(A) &\approx -1.818 \end{aligned}$$

Under this policy, the agent expects about -1.82 in total return from state A. The negative value reflects the -1 cost per step and the 50% chance of looping. If the policy always went right, the value would be simply -1: one step, one reward, done.



👉 For larger state spaces, the circular dependencies become a system of simultaneous equations. Iterative methods (covered later) handle these efficiently without explicit matrix inversion.

Now let's go ahead and understand the Bellman expectation equation for q_π.

The Bellman expectation equation for q_π

The same recursive logic applies to the action-value function. Recall that $q_\pi(s, a)$ is the expected return from taking action a in state s and then following π forever after.

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

We introduced q_π alongside v_π in Part 2.

The difference is small but important.

- With $v_\pi(s)$, the agent picks its first action from π like any other step.
- With $q_\pi(s, a)$, we force the first action to be a and only follow π from the next step onward.

This is important because if you know q_π for every action, picking the best one is trivial. You can just take $\arg\max_a q_\pi(s, a)$ and no model of the environment is needed.

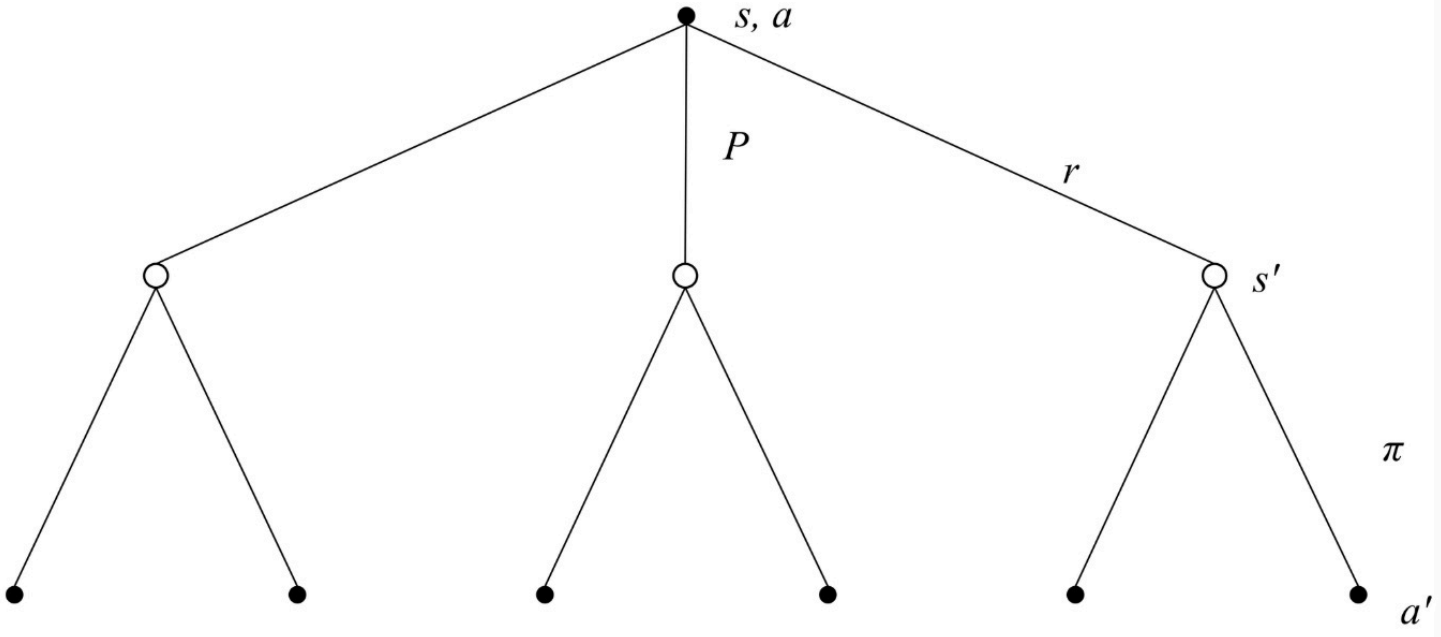
The Bellman expectation equation for q_π is:

$$q_\pi(s,a) = \sum_{s'} P(s'|s,a)\Big[R(s,a,s') + \gamma \sum_{a'} \pi(a'|s')\, q_\pi(s',a')\Big]$$

Here,

- $q_\pi(s,a)$ is the value of taking action a in state s under π.
- The outer sum runs over next states s', weighted by the transition probability $P(s'|s,a)$.
- Inside the brackets, $R(s,a,s')$ is the immediate reward.
- The inner sum runs over actions a' the agent might take in the successor state s', weighted by $\pi(a'|s')$, multiplied by $q_\pi(s',a')$.

The structure mirrors the v_π equation but starts one step later. Since the first action is already pinned to a, the first layer averages over the environment's transition. The second layer, inside s', averages over the policy's next action choice.



Backup diagram for q_π

Now, as we know from chapter 2, the two value functions are linked as:

$$v_\pi(s) = \sum_a \pi(a \mid s)\, q_\pi(s, a)$$

We derived this relationship in Part 2. It says something intuitive that the value of a state is just the weighted average of the action values, where the weights come from the policy.

Think of v_π as the overall grade for a state, and q_π as the grade for each individual action you could take there.

Moving on, the value of a state is the policy-weighted average of the action values. And from the Bellman equation for q_π, we can also write:

$$q_\pi(s,a) = \sum_{s'} P(s'|s,a)\Big[R(s,a,s') + \gamma\, v_\pi(s')\Big]$$

These two relationships interlock: v_π can be expressed in terms of q_π, and q_π in terms of v_π. This interlocking structure will become important when we define optimality.

Optimal value functions and the Bellman optimality equation

So far, the Bellman equations describe the value of following a specific policy π. But the agent's goal is not to evaluate one policy. It is to find the best one. This brings us to the concept of optimal value functions.

Optimal policy and optimal value functions

A policy π is defined to be better than or equal to policy π' if $v_\pi(s) \geq v_{\pi'}(s)$ for all states s. An optimal policy, written π_*, is one that is at least as good as every other policy.

For finite MDPs, at least one optimal policy always exists. There may be several, but they all share the same value functions.

Up to this point, both in Part 2 and earlier in this chapter, every value we have looked at has been tied to one specific policy π. Change the policy, and the values change.

The natural next question is → what if we could pick the best policy? What is the highest value any state could ever have?

The optimal state-value function v_* is defined as:

$$v_*(s) = \max_\pi v_\pi(s) \quad \text{for all } s \in S$$

$v_*(s)$ is the maximum expected return achievable from state s over all possible policies. Similarly, the optimal action-value function q_* is:

$$q_*(s, a) = \max_\pi q_\pi(s, a) \quad \text{for all } s \in S,\ a \in A$$

$q_*(s, a)$ is the maximum expected return from taking action a in state s and behaving optimally thereafter.

👉 Once you know q_*, the optimal policy is trivial. Just pick:

$$\arg\max_a q_*(s, a)$$

in every state. No model needed, no lookahead needed. This is why estimating q_* is the central goal of many RL algorithms.

The Bellman optimality equation for v_*

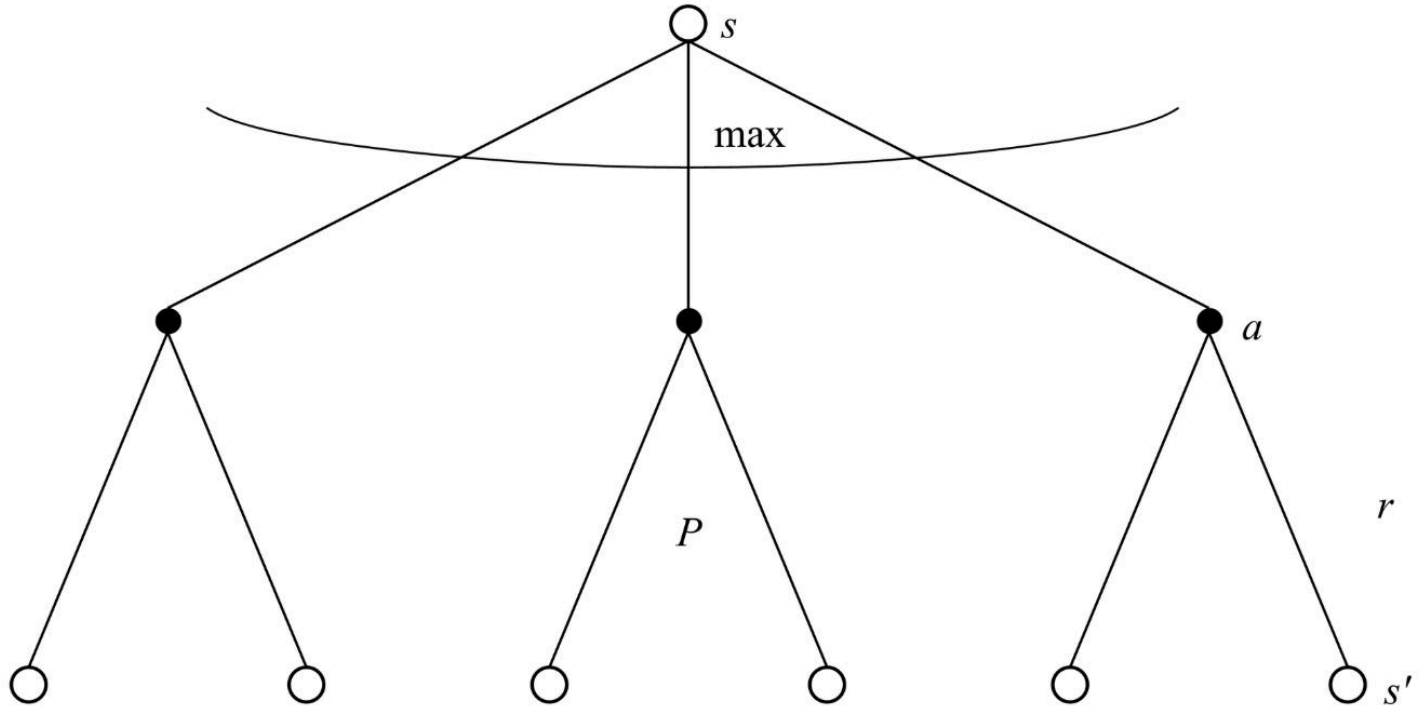
Because an optimal policy selects the best action in every state, the Bellman equation for v_* replaces the policy-weighted average with a maximum over actions.

The Bellman optimality equation for v_* is:

$$v_*(s) = \max_a \sum_{s'} P(s'|s,a)\Big[R(s,a,s') + \gamma\, v_*(s')\Big]$$

Let us unpack this:

- $v_*(s)$ is the optimal value of state s.
- $\max_a$ selects the action that yields the highest value.



Backup diagram for v_*

👉 The key structural difference: the Bellman expectation equation is linear (weighted sum over actions), so it can be solved by linear algebra. The Bellman optimality equation is non-linear (because of the max), so it cannot. This is why we need iterative algorithms.

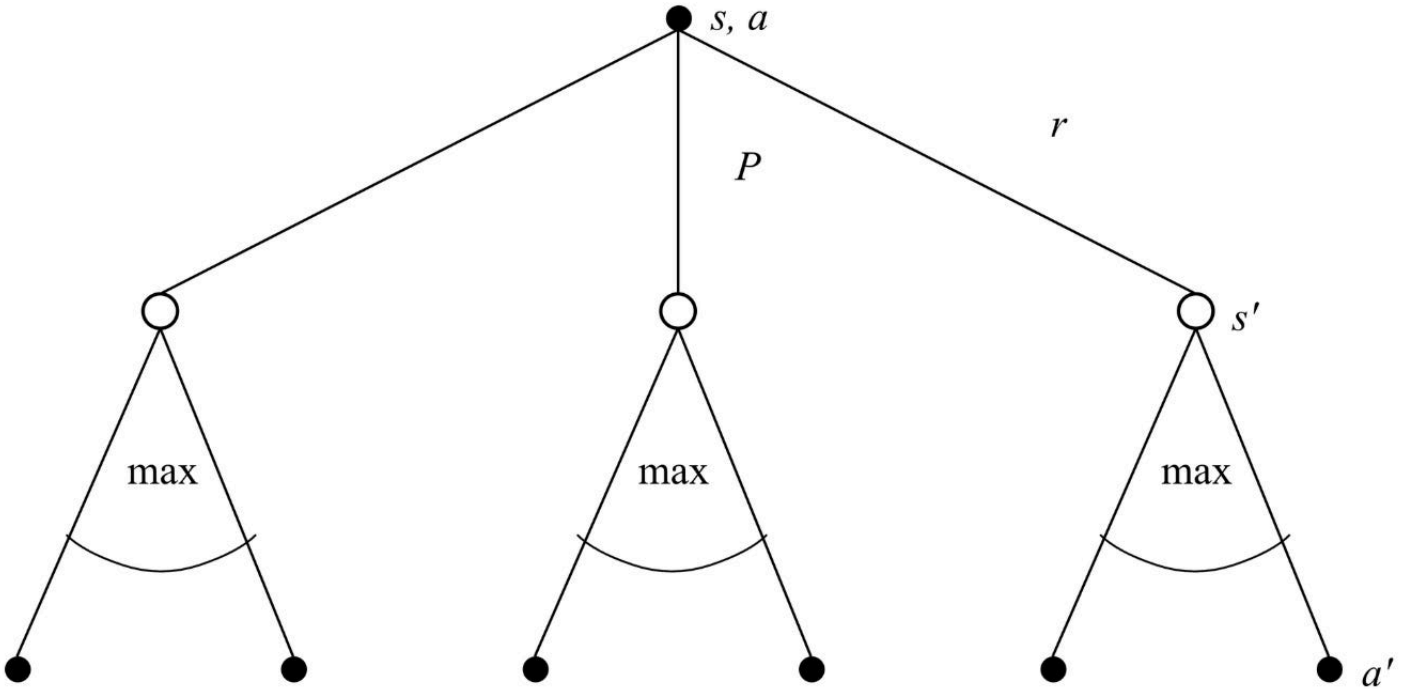
The Bellman optimality equation for q_*

The Bellman optimality equation for q_* is:

$$q_*(s,a) = \sum_{s'} P(s'|s,a)\Big[R(s,a,s') + \gamma \max_{a'} q_*(s',a')\Big]$$

Here,

- $q_*(s,a)$ is the optimal value of taking action a in state s.
- Sum runs over next states.
- $\gamma \max_{a'} q_*(s',a')$ is the discounted optimal value from the successor, obtained by taking the best action a' in s'.



Backup diagram for q_*

The relationship between v_* and q_* is:

$$v_*(s) = \max_a q_*(s, a)$$

The optimal value of a state is the value of the best action in that state.

👉 Solving the Bellman optimality equations gives v_* and q_* directly, from which the optimal policy follows immediately. The entire RL problem, in principle, reduces to solving these equations. The difficulty is that solving them exactly requires a complete model and becomes computationally intractable for large state spaces.

In summary, we now have four Bellman equations: two expectation equations (for v_π and q_π) and two optimality equations (for v_* and q_*).

The expectation equations describe what happens under a given policy. The optimality equations describe the best possible behavior. DP algorithms exploit

these equations to compute optimal policies.

From equations to algorithms: dynamic programming

The Bellman equations are mathematical identities. They describe relationships that the true value functions satisfy. But they are not algorithms. Dynamic programming turns them into iterative update rules that converge to the solution.

DP requires a complete and accurate model of the environment: the full transition function P and reward function R for every state-action pair. This places DP firmly in the model-based setting.

The core idea of DP is simple: use the Bellman equations as update rules. Start with an arbitrary estimate of the value function. Sweep through all states, updating each one according to the Bellman equation. Repeat until the estimates converge.

There are four components we will cover:

- policy evaluation (compute v_π for a given π)
- policy improvement (derive a better π from v_π)
- policy iteration (alternate the two until convergence)
- value iteration (combine them into a single update)

Policy evaluation

Policy evaluation answers the question: given a fixed policy π, what is v_π? This is also called the prediction problem, since we are predicting how much reward the policy will accumulate.

The iterative update

The Bellman expectation equation for v_π is:

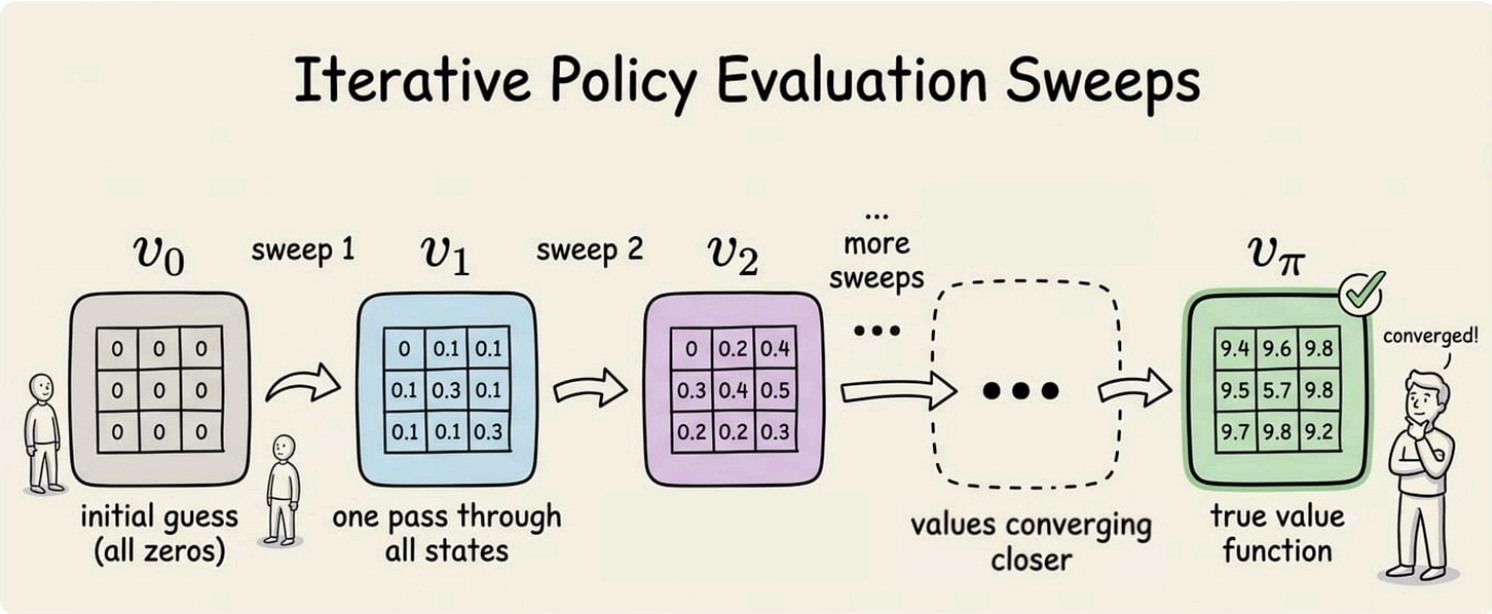
$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) \Big[R(s,a,s') + \gamma \, v_\pi(s') \Big]$$

We turn this into an update rule. Start with an arbitrary initial estimate v_0 (typically all zeros). At each sweep k, update every state:

$$v_{k+1}(s) \leftarrow \sum_{a} \pi(a|s) \sum_{s'} P(s'|s,a) \Big[R(s,a,s') + \gamma\, v_k(s') \Big]$$

The left side is the new estimate. The right side uses the old estimates from the previous sweep.

> 👉 One full pass through all states is called a sweep.



This update is a contraction mapping, i.e., the distance between v_k and the true v_π shrinks by a factor of at most γ with every sweep.

> 💡 Think of a contraction mapping like repeatedly squeezing a spring. No matter how far you stretch it initially, each squeeze brings it closer to rest.
>
> Here, the "rest position" is the true v_π, and each sweep of the Bellman update squeezes the gap by a factor of at most γ. Since $\gamma < 1$, the gap keeps shrinking and the estimates eventually settle on the correct values.
>
> This guarantee comes from a result in mathematics called the Banach fixed-point theorem.

Because $\gamma < 1$ (or the task is episodic), the iterates converge to the unique fixed point v_π as $k \to \infty$.

In practice, we stop when the maximum change across all states in a single sweep falls below a small threshold θ:

$$\max_s \left| v_{k+1}(s) - v_k(s) \right| < \theta$$

Algorithm

The full algorithm for iterative policy evaluation is:

1. Initialize $v(s) = 0$ for all s.
2. Repeat:
    - For each state s:
        - $v_{\text{new}}(s) \leftarrow \sum_a \pi(a|s) \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma\, v(s')]$
    - If $\max_s |v_{\text{new}}(s) - v(s)| < \theta$, stop.
    - $v \leftarrow v_{\text{new}}$
3. Return $v \approx v_\pi$.

The convergence rate depends on γ. With γ close to 1, convergence is slow because the contraction factor is close to 1. With γ close to 0, convergence is fast but the resulting value function only reflects near-term rewards.

👉 The difference between the Bellman equation and the update rule is subtle but crucial. The equation is a condition that the true v_π satisfies. The update rule is an iterative algorithm that converges to v_π. The equation uses v_π on both sides. The update uses v_k on the right and produces v_{k+1} on the left.

Policy improvement

Policy evaluation gives us v_π for a specific policy. The next question: can we use v_π to find a better policy?

The greedy policy

Given v_π, we construct a new policy π' by acting greedily. For each state s, pick the action that maximizes the expected one-step lookahead:

$$\pi'(s) = \arg\max_a \sum_{s'} P(s'|s,a)\Big[R(s,a,s') + \gamma\, v_\pi(s')\Big]$$

💡 Notice what this arg max needs the transition probabilities $P(s'|s,a)$ and the reward $R(s,a,s')$.

These are the same model components from the MDP 5-tuple (S, A, P, R, γ) we defined in Part 2. Without knowing them, we cannot do this lookahead.

That is what makes this a model-based operation.

Later in the series, when we move to model-free methods, we will sidestep this by working with q_π directly, which (as we saw in Part 2) absorbs the dynamics into itself so the agent never needs to know P.

The phrase "one-step lookahead" means we only optimize the current action, then assume π takes over from the next state onwards. a is the action chosen now. This is the one step we're optimizing.

So the full computation is: take action a once, then follow π for the rest of time. The "step" being looked at is just the current one; everything after is handled by π (baked into v_π).

Notice that the expression inside the arg max is exactly $q_\pi(s, a)$. So the greedy policy simply selects in every state:

$$\arg\max_a q_\pi(s, a)$$

The policy improvement theorem

Above we said "act greedily and you'll get a better policy", but how do we know that's true? Could being greedy for one step somehow backfire later? The policy improvement theorem answers this: greedy improvement is guaranteed to help, never hurt.

In simple words, the theorem states: If, in every state s, π' 's action is at least as good as π's action (assuming we follow π afterwards), then π' is at least as good as π everywhere, even when used throughout.

Formally: if $q_\pi(s, \pi'(s)) \geq v_\pi(s)$ for every state s, then $v_{\pi'}(s) \geq v_\pi(s)$ for every state s.

👉 When does improvement stop? When the greedy policy is the same as the current policy: $\pi' = \pi$. In that case:

$$v_\pi(s) = \max_a q_\pi(s, a)$$

for all states, which is exactly the Bellman optimality equation. The policy is optimal. So $v_\pi = v_*$ and $\pi = \pi_*$.

Policy iteration

Policy iteration combines evaluation and improvement into a loop. Start with any policy, evaluate it, improve it and repeat. The result is the optimal policy.

1. Initialize with an arbitrary deterministic policy π_0.
2. Repeat:
    - **Policy evaluation:** Compute v_{π_k} by iterating the Bellman expectation equation until convergence.
    - **Policy improvement:** Set $\pi_{k+1}(s) = \arg\max_a q_{\pi_k}(s, a)$ for all s.
    - If $\pi_{k+1} = \pi_k$, stop.
3. Return $\pi_k = \pi_*$ and $v_{\pi_k} = v_*$.

👉 Policy iteration is sometimes called "exact policy iteration" to distinguish it from methods that truncate the evaluation step.

The cost of policy iteration lies in the evaluation step. Each evaluation runs the iterative update until convergence, which can require many sweeps, especially when γ is close to 1.

This raises a question: do we really need exact evaluation before improving? The answer is no, and this observation leads to value iteration.

Value iteration

Value iteration makes a simple observation: we do not need to wait for policy evaluation to converge. We can truncate it to a single sweep and fold the improvement step directly into the update.

The update rule

Value iteration combines the Bellman optimality equation into a single iterative update:

$$v_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)\Big[R(s,a,s') + \gamma\, v_k(s')\Big]$$

Compare this to the policy evaluation update, which uses a policy-weighted sum. Value iteration replaces that sum with a max. Each sweep simultaneously evaluates and improves, because the max selects the best action at each state.

Policy iteration typically requires fewer outer iterations (improvement steps), but each iteration is expensive because evaluation runs to convergence. Value iteration requires more outer iterations (sweeps), but each is cheap: a single pass through all states.

Which is faster depends on the problem. For small problems, policy iteration often converges in very few improvement steps. For larger problems, value iteration can be more practical because it avoids the cost of full evaluation.

👉 Both algorithms converge to the same v_* and π_*. They differ only in how they get there.

Note: After value iteration converges, we still need to extract the policy. We do this for each state by computing:

$$\pi_*(s) = \arg\max_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma\, v_*(s')]$$

This is a single pass, not an iterative process.

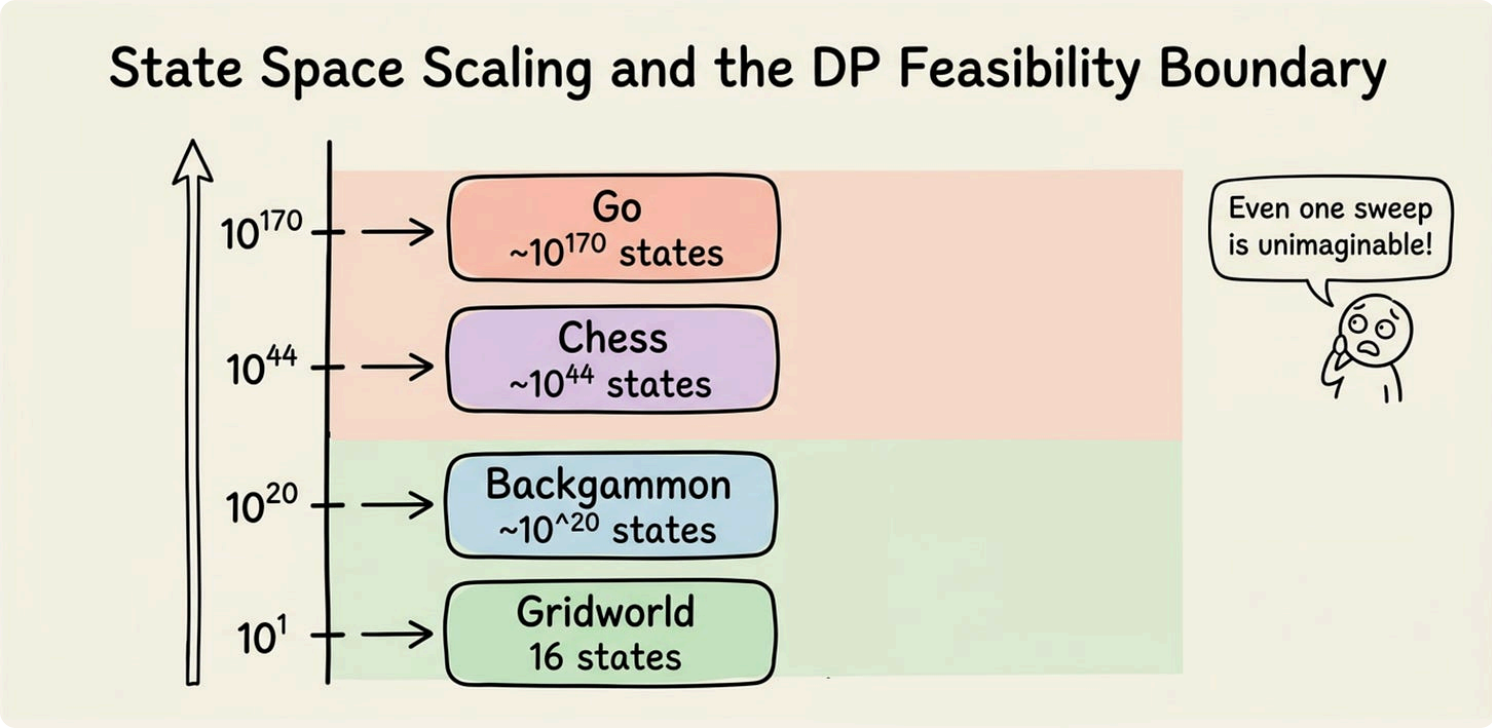
Now that we have a reasonable grasp of DP, let's examine one of its key shortcomings beyond being model-based.

The curse of dimensionality in DP

Even with a model in hand, DP runs into a computational wall. Bellman himself called this the "curse of dimensionality" (Bellman, 1957). The problem is that DP

sweeps every state on every iteration, and the number of states grows exponentially with the number of state variables.

Consider a problem with d state variables, each taking n values. The total state count is n^d. For example, 2D gridworld has two state variables (x, y) and if each has four values, that makes total states as $4^2 = 16$. Similarly, with $d = 10$ and $n = 100$, that gives a huge 10^{20} count of states.



Thus, iterations and sweeps at such a huge scale becomes unimaginable.

So what does DP give us?

Despite its limits, DP is foundational and also serves as a benchmark. For small problems where it is tractable, it gives the exact optimal solution, against which approximate methods can be measured.

Even with the curse of dimensionality, DP is far more efficient than brute-force search over policies. The number of deterministic policies is $|A|^{|S|}$, astronomically larger than $|S|$ for all but the smallest problems.

In summary, DP is the gold standard for small, known environments. It gives exact answers through clean algorithms grounded in the Bellman equations. Its practical limits, the need for a model and the curse of dimensionality, push us toward model-free methods which we'll be studying later in the series.

The model-free methods we will study later drop the need for P and R entirely. The agent learns from experience alone, similar to how we used Monte Carlo rollouts in Part 2 to estimate v_π without ever writing down the transition function. The key difference is that the upcoming methods will be far more sample-efficient than running thousands of episodes and averaging.

We have now understood all the key concepts for this chapter. The pieces are in place. So let's go ahead and dive into hands-on section for some practical experimentation.

Hands-on: policy iteration and value iteration on a 4×4 gridworld

Here we will implement both policy iteration and value iteration on a 4×4 gridworld and compare the results.

Setup

The code and project setup are attached below as a zip file. You can extract it and run `uv sync` to get going.

```
.python-version
dp.py
gridworld.py
main.py
pyproject.toml
uv.lock
```

Download the zip file below:

dp-gridworld

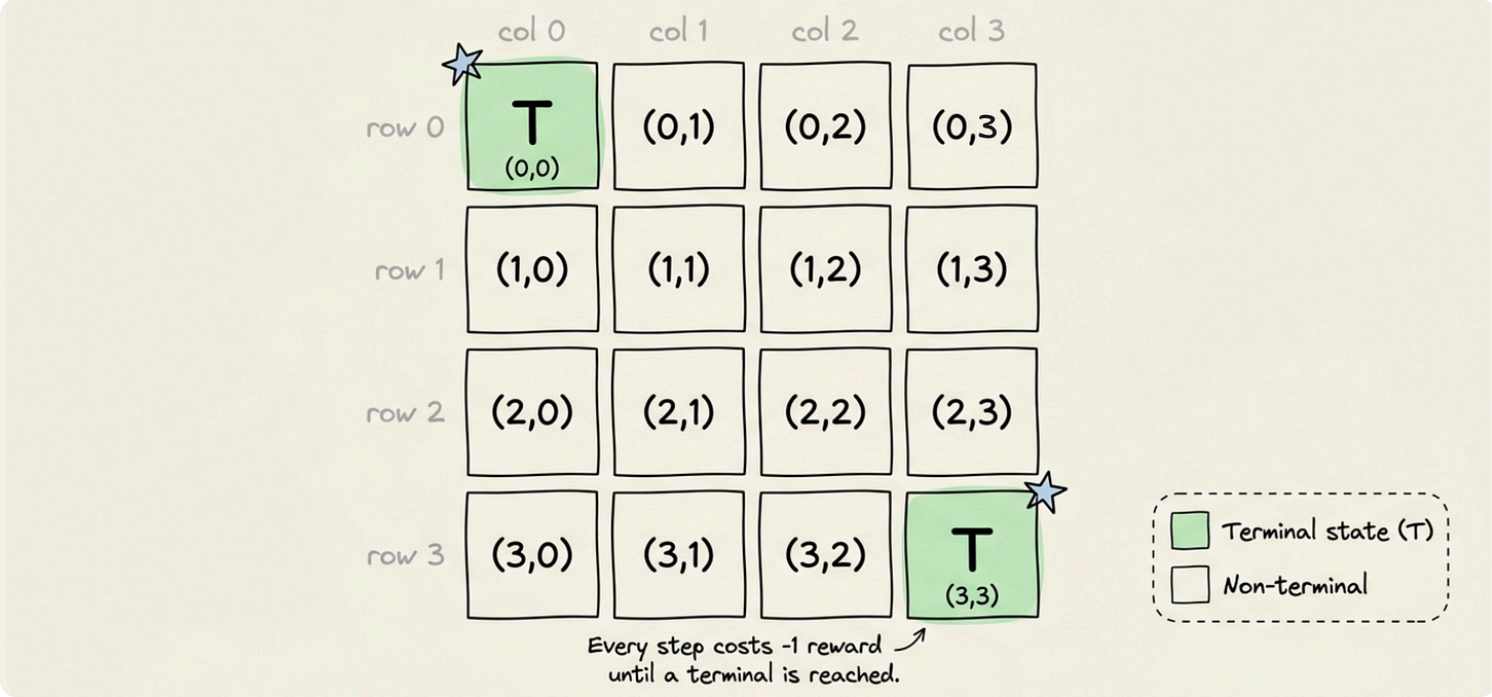
dp-gridworld.zip • 40 KB

For details about versions and dependencies, check the `.python-version` and `pyproject.toml` files.

👉 It is recommended to follow the explanation ahead side-by-side with the above attached project for a more comprehensive understanding.

The environment

The grid has 16 cells. Two are terminal: $(0, 0)$ (top-left) and $(3, 3)$ (bottom-right). The remaining 14 are non-terminal. Every transition gives reward -1. Transitions are deterministic: the agent moves in the chosen direction unless it would leave the grid, in which case it stays in place. We also use $\gamma = 0.99$ to show discounting.



Now let's look at the code:

The GridWorld class is nearly identical to the one in Chapter 2, with one change: we removed the `reset` method and the episode-running logic. DP does not need to simulate episodes. It works directly with the transition function. The `step` method serves as that function: given a state and action, it returns the deterministic next state and reward.

A few details about the implementation:

- States are stored as flat indices 0 through 15. The `state_to_rc` and `rc_to_state` methods convert between flat indices and (row, column) coordinates.
- Terminal states return reward 0 and transition to themselves. Non-terminal states always return reward -1.
- Wall collisions are handled by clamping: if the move would go off the grid, the agent stays in place.

The DP algorithms

Dynamic programming algorithms for the gridworld: iterative policy evaluation, policy improvement, policy iteration and value iteration.

First let's take a look at policy evaluation and improvement:

Let us walk through each one:

- `policy_evaluation` implements the iterative Bellman update for a fixed deterministic policy:
  - It initializes V to zeros, then sweeps through all non-terminal states.
  - For each state, it applies the Bellman update: $V(s) \leftarrow R + \gamma V(s')$, where s' is the single next state under the deterministic policy.
  - The function tracks the maximum change (`delta`) across each sweep and stops when it falls below `theta`.
  - It returns the converged value function and the number of sweeps.

> 👉 Because the transitions are deterministic and the policy is deterministic, the inner sum over next states collapses to a single term, which is why the update is a simple assignment rather than a weighted sum.

- `policy_improvement` constructs the greedy policy with respect to a given value function. For each non-terminal state, it computes the one-step lookahead value for all four actions (up, right, down, left) and picks the action with the highest value. This is the arg max operation from the theory.

Now let's take a look at policy iteration and value iteration:

Here:

- `policy_iteration` glues evaluation and improvement together. It starts with the all-up policy (action 0 for every state), evaluates it, improves it, and

repeats until the policy stops changing. It tracks the total number of evaluation sweeps across all iterations.

- `value_iteration` implements the Bellman optimality update. Each sweep applies $V(s) \leftarrow \max_a [R + \gamma V(s')]$ for every non-terminal state. After convergence, it extracts the greedy policy in a single final pass.

Running the experiment

Finally, let's instantiate the gridworld and run the experiment:

The main script instantiates the gridworld with $\gamma = 0.99$, runs both algorithms, and compares the results. It also generates a side-by-side visualization with value heatmaps and policy arrow plots.

Results

Running `python main.py` produces the following output:

```
================================================
POLICY ITERATION
================================================
Policy improvement steps: 4
Total evaluation sweeps:  5506

Optimal value function (policy iteration):
----------------------------------------
    0.00    -1.00    -1.99    -2.97
   -1.00    -1.99    -2.97    -1.99
   -1.99    -2.97    -1.99    -1.00
   -2.97    -1.99    -1.00     0.00

Optimal policy (policy iteration):
----------------------------------------
   T    ←    ←    ↓
   ↑    ↑    ↑    ↓
   ↑    ↑    →    ↓
   ↑    →    →    T

================================================
VALUE ITERATION
================================================
Sweeps to convergence: 4

Optimal value function (value iteration):
----------------------------------------
```

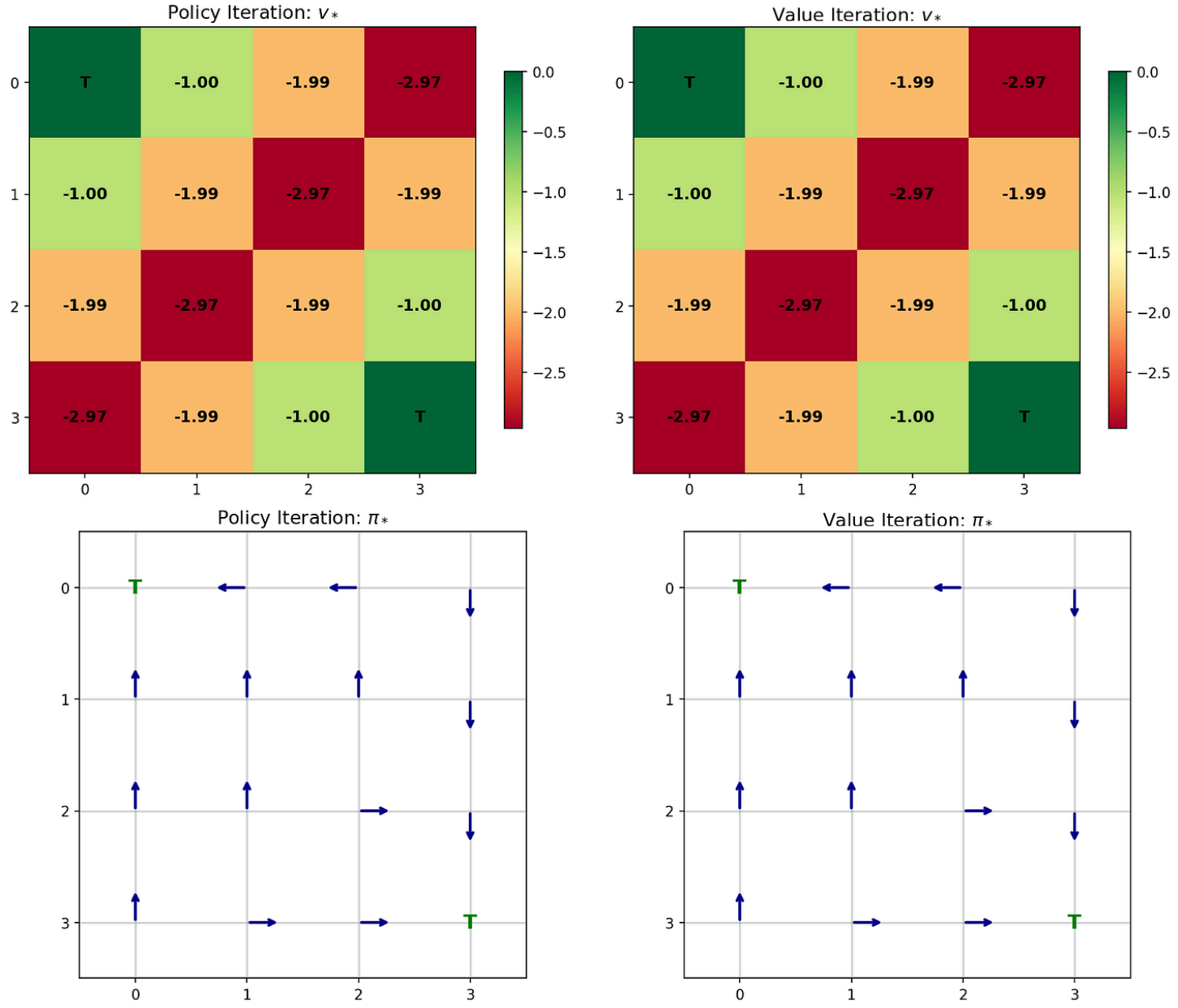
```
  0.00    -1.00    -1.99    -2.97
 -1.00    -1.99    -2.97    -1.99
 -1.99    -2.97    -1.99    -1.00
 -2.97    -1.99    -1.00     0.00

Optimal policy (value iteration):
----------------------------------------
  T    ←    ←    ↓
  ↑    ↑    ↑    ↓
  ↑    ↑    →    ↓
  ↑    →    →    T

================================================
COMPARISON
================================================
Max |V_pi - V_vi|: 0.00e+00
Policies identical: True

Saved plot to dp_comparison.png
```

Several observations from these results:

- Both algorithms converge to the same optimal value function and the same optimal policy. The max difference between their value functions is exactly zero, and the policies are identical. This confirms the theory: both algorithms find the same v_* and π_*.

- The optimal policy is intuitive. Every state points toward the nearest terminal.
- The iteration counts tell a revealing story:
    - Policy iteration needed only 4 improvement steps to converge, but those 4 steps required 5,506 total evaluation sweeps. The evaluation phase dominates the cost, because $\gamma = 0.99$ makes convergence slow.
    - Value iteration, by contrast, converged in just 4 sweeps. This is because the gridworld is small and has short optimal paths (at most 6 steps from the farthest state to a terminal). The max operator propagates optimal values faster than the policy-weighted average.

A plot is also saved that visualizes the comparison:



The heatmaps show the symmetric structure of v_*: the grid is symmetric about the diagonal from $(0, 0)$ to $(3, 3)$. The policy arrows confirm that every state routes the agent toward the nearest terminal.

That wraps up the hands-on section. With this we conclude the discussion for this chapter. In the upcoming chapters, we will continue to build on the core ideas of RL, and explore concepts and their implementations wherever applicable.

Conclusion

In this chapter, we explored the recursive structure at the heart of reinforcement learning.

We looked at the Bellman expectation equations for v_π and q_π, showing that the value of a state decomposes into the immediate reward plus the discounted value of the successor. We then saw the Bellman optimality equations for v_* and q_*, which replace the policy average with a max over actions, characterizing the best achievable performance.

We studied four DP components:

- Policy evaluation iteratively computes v_π for a given policy.
- Policy improvement constructs a greedy policy from v_π, guaranteed to be at least as good by the policy improvement theorem.
- Policy iteration alternates the two until convergence to π_*.
- Value iteration merges evaluation and improvement into a single update, using the Bellman optimality equation directly.

We examined the limitations of DP: it requires a complete model of the environment and faces the curse of dimensionality for large state spaces. These constraints motivate the model-free methods.

Finally, we built a concrete implementation of both policy iteration and value iteration on the 4×4 gridworld and saw them converge to the same optimal policy and value function, while differing in computational cost.

In the next chapter, we will move beyond the model-based assumption and explore model-free learning.

The aim, as always, is to build a strong conceptual foundation and equip you with a flexible framework for reasoning about the core principles and the broader landscape in general.

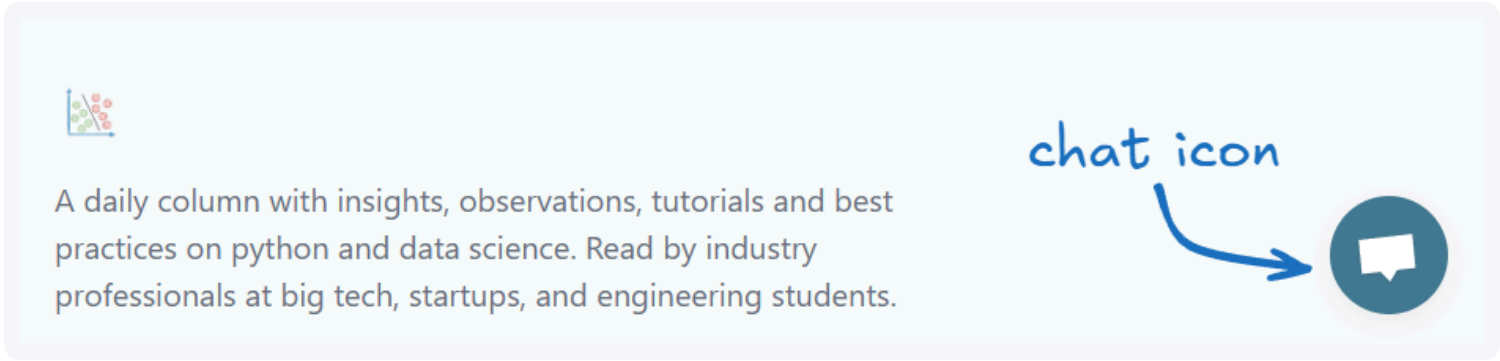
As always, thanks for reading!

Any questions?

Feel free to post them in the comments.

Or

If you wish to connect privately, feel free to initiate a chat here:



Published on May 10, 2026

Comments Share

Previous
Markov Decision Processes and Value Functions

Next
Model-Free Learning

A daily column with insights, observations, tutorials and best practices on python and data science. Read by industry professionals at big tech, startups, and engineering students.

Menu

Contact

FAQ